CINEFLEX

Estrategia de Pruebas

Taller Práctico Integrador: QA en Producción · Fase A

Elaborado por: Harrisson Montaña Hurtado

Programa: Ingeniería de Sistemas

Asignatura: Pruebas de software

Fecha: Octubre de 2026

Mocoa, Colombia

Contenido

1. Introducción

2. Alcance

3. Arquitectura bajo prueba

4. Ambiente y herramientas

5. Reto 1: Pruebas de integración con Stubs y Drivers

6. Reto 2: Pruebas de sistema y regresión

7. Reto 3: Comparativa de herramientas

8. Plan de ejecución de pruebas

9. Resultados obtenidos

10. Riesgos y mitigación

11. Criterios de entrada y salida

12. Conclusiones

13. Anexos

1. Introducción

Este documento define la estrategia de aseguramiento de calidad para **CineFlex**, una plataforma web de venta de boletos de cine que debe operar sin fallas durante la noche del gran estreno mundial. Aquí se documentan las decisiones arquitectónicas de QA tomadas antes de escribir el código de automatización: cómo integrar los módulos cuando un tercero no está disponible, qué requisitos se verifican, cómo se seleccionan las pruebas de regresión y qué herramienta usar en cada escenario.

1.1 Objetivo general

Certificar que CineFlex cumple sus requisitos funcionales y no funcionales en las capas de frontend, backend y servicios de terceros, mediante pruebas automatizadas repetibles e integradas a un pipeline de integración continua.

1.2 Objetivos específicos

- Validar la integración del backend con la pasarela de pagos mediante un **Stub**, sin depender del banco.
- Automatizar las pruebas de API (login, cartelera y reservas) con **Postman** y ejecutarlas en la nube con **Newman**.
- Automatizar los flujos críticos de la interfaz con **Selenium WebDriver** aplicando el patrón **Page Object Model**.
- Demostrar con **Apache JMeter** que la plataforma soporta 500 usuarios concurrentes.
- Bloquear la integración de código que rompa las pruebas mediante **GitHub Actions**.

2. Alcance

Dentro del alcance	Fuera del alcance
Autenticación con JWT y bloqueo tras 3 intentos fallidos	Integración con la API real del banco (se usa Stub)
Consulta de cartelera y de asientos disponibles	Persistencia en base de datos real (datos en memoria)
Flujo de compra: película → asiento → pago → ticket	Pruebas en dispositivos móviles nativos
Navegación segura (cerrar sesión y botón "Atrás")	Pruebas de penetración avanzadas
Cabeceras de seguridad (Helmet) y trazabilidad (Morgan)	Usabilidad y accesibilidad formal
Carga de 500 usuarios concurrentes	
Pipeline de pruebas en cada push	

3. Arquitectura bajo prueba

CineFlex se compone de tres capas. Para el taller se construyó un entorno simulado (mock) de cada una, de modo que las pruebas se ejecuten de forma controlada y repetible.

Capa	Componente	Responsabilidad
Frontend (UI)	frontend/index.html y styles.css	Vistas de login, cartelera, asientos, pago y ticket. Cada elemento tiene un ID estable para Selenium. El enrutamiento por hash incluye una guardia de sesión.
Backend (API REST)	backend/server.js (Node.js + Express)	Endpoints /api/login, /api/cartelera, /api/asientos/:id y /api/reservas. Protegido con Helmet, trazado con Morgan y autenticado con JWT.
Terceros	pasarelaBancariaStub()	Simula la respuesta de la pasarela de pagos del banco mientras no hay acceso a su entorno de pruebas.

3.1 Endpoints de la API

Método	Ruta	Autenticación	Respuestas esperadas
POST	/api/login	No	200 con token · 400 datos faltantes · 401 credenciales inválidas
GET	/api/cartelera	Bearer JWT	200 lista de películas · 401 sin token
GET	/api/asientos/:peliculaId	No	200 mapa de asientos · 404 película inexistente
POST	/api/reservas	Bearer JWT	201 ticket · 400 incompleto · 402 pago rechazado · 409 asiento ocupado

4. Ambiente y herramientas

Herramienta	Uso en el proyecto	Ubicación
Node.js 20 + Express	Backend mock y servidor del frontend	backend/
Helmet / Morgan	Seguridad de cabeceras HTTP / registro de peticiones	backend/server.js
Postman + Newman	Pruebas automatizadas de API (local y CI)	api_tests/
Selenium WebDriver + Mocha	Pruebas de UI con Page Object Model	ui_tests/
Apache JMeter 5.6	Pruebas de carga y estrés	performance/
GitHub Actions	Continuous Testing en cada push	.github/workflows/

Datos de prueba: usuario cliente@cineflex.com / Cine123*. Tarjetas aprobadas por el Stub: 4111111111111111 y 5555555555554444. Cualquier otro número es rechazado.

5. Reto 1: Pruebas de integración con Stubs y Drivers

5.1 Situación

El equipo de frontend ya terminó la interfaz de pagos y el backend ya tiene listo el endpoint POST /api/reservas. Sin embargo, el banco todavía no da acceso a su API en el entorno de pruebas. Esperar al banco bloquearía la certificación de la funcionalidad más importante del negocio: vender boletos.

5.2 Enfoque elegido: Top-Down

Se elige integración **Top-Down** porque los módulos de nivel superior (UI de pagos y endpoint de reservas) ya existen y el módulo faltante es el de más bajo nivel: el banco. En Top-Down se integra desde la capa de presentación hacia abajo y cada componente inferior ausente se reemplaza por un **Stub**.

El enfoque Bottom-Up no aplica en este caso: se usaría si tuviéramos el módulo inferior terminado (el banco) y faltara la capa superior. Entonces haría falta un **Driver**, es decir, un programa que invoque al módulo inferior simulando a quien lo llama.

Aspecto	Stub (Top-Down)	Driver (Bottom-Up)
Qué reemplaza	Un módulo llamado que aún no existe (el banco)	Un módulo que llama y aún no existe (la UI o el controlador)
Dirección	Es invocado por el código bajo prueba	Invoca al código bajo prueba
En CineFlex	pasarelaBancariaStub() responde en lugar del banco	No se necesita: la UI y la API ya existen
Analogía	El doble de acción que actúa mientras llega el actor	El director que da las órdenes para ensayar una escena suelta

5.3 Diseño del Stub de la pasarela bancaria

- **Contrato idéntico al del banco:** recibe un objeto tarjeta {titular, numero, vencimiento, cvv} y devuelve {aprobada, codigoAutorizacion} o {aprobada: false, motivo}.
- **Respuestas deterministas:** solo las tarjetas de prueba 4111111111111111 (Visa) y 5555555555554444 (Mastercard) se aprueban. Así cada caso tiene un resultado esperado conocido de antemano.
- **Punto único de reemplazo:** el Stub es una sola función. Cuando el banco entregue su sandbox, se sustituye por la llamada HTTP real sin modificar el endpoint ni el frontend.
- **Traducción a HTTP:** aprobado → 201 Created con ticket; rechazado → 402 Payment Required con el mensaje "Pago rechazado: tarjeta inválida"; datos incompletos → 400 Bad Request.

```
// backend/server.js
function pasarelaBancariaStub(tarjeta) {
  const TARJETAS_APROBADAS = ['4111111111111111', '5555555555554444'];
```

```
  if (tarjeta && TARJETAS_APROBADAS.includes(tarjeta.numero)) {
    return { aprobada: true, codigoAutorizacion: 'AUTH-' + Date.now() };
  }
  return { aprobada: false, motivo: 'Tarjeta rechazada por el banco' };
}

// Dentro de POST /api/reservas
const pago = pasarelaBancariaStub(tarjeta);
if (!pago.aprobada) {
  return res.status(402).json({ error: 'Pago rechazado: tarjeta inválida' });
}
```

5.4 Cómo se valida hoy mismo

Prueba	Herramienta	Qué demuestra
Reserva con tarjeta falsa	Postman / Newman	El backend responde 402, con el mensaje exacto y sin emitir ticket.
CP12: Tarjeta falsa rechazada	Selenium	La UI muestra el error devuelto por el Stub.
CP13: Compra End-to-End	Selenium	Con tarjeta aprobada se emite el ticket virtual con el asiento elegido.

5.5 Riesgos residuales del uso del Stub

El Stub no reproduce la latencia, los timeouts ni los códigos de error reales del banco. Cuando el banco entregue su entorno de pruebas se ejecutará una **prueba de contrato** para confirmar que su respuesta coincide con la que asume el Stub, y se agregarán casos de timeout y reintento.

6. Reto 2: Pruebas de sistema y regresión

6.1 Requisitos funcionales y no funcionales

ID	Tipo	Requisito	Criterio de aceptación	Verificación
RF-01	Funcional	El usuario debe autenticarse con correo y contraseña; tras 3 intentos fallidos consecutivos se bloquea el inicio de sesión.	Login válido → vista "Cartelera". Al tercer fallo, botón deshabilitado y mensaje de bloqueo.	Selenium CP02–CP05 Postman Login
RF-02	Funcional	El usuario autenticado debe poder elegir película y asiento libre, pagar y recibir un ticket virtual.	Pago aprobado → ticket con ID y asiento. Tarjeta inválida → error 402 visible.	Selenium CP09–CP13 Postman Reserva
RNF-01	Rendimiento	La consulta de cartelera debe responder en menos de 300 ms.	responseTime < 300 ms en cada ejecución.	Postman Cartelera

ID	Tipo	Requisito	Criterio de aceptación	Verificación
RNF-02	Carga / Seguridad	La consulta de asientos debe soportar 500 usuarios concurrentes y toda respuesta debe incluir cabeceras de seguridad.	Error < 1 % con 500 VUs y ramp-up de 15 s. Cabecera X-Content-Type-Options: nosniff.	JMeter Postman Login

6.2 Estrategia de regresión: selectiva basada en riesgo e impacto

Caso planteado: un desarrollador cambia el color de un botón secundario en la vista del perfil.

Decisión: no se vuelve a ejecutar toda la suite de la pasarela de pagos. El cambio es solo visual (CSS) y está aislado en una vista que no participa en el flujo de compra. No modifica /api/reservas, el Stub, el formulario de pago ni el JWT, así que la probabilidad de afectar los pagos es prácticamente nula. Correr la suite completa gastaría minutos de CI y tiempo del equipo sin aportar información nueva.

En su lugar se ejecutan: las pruebas de UI de la vista afectada y un **smoke test** de 3 casos críticos (CP01 carga del login, CP05 login exitoso y CP13 compra E2E) que confirma que el sistema sigue vivo.

Criterio de selección de casos:

Tipo de cambio	Riesgo	Pruebas que se ejecutan
Estilos o CSS de una vista no crítica	Bajo	UI de esa vista + smoke test (CP01, CP05, CP13)
Login, JWT o guardia de sesión	Alto	Suite de login y navegación segura (CP01–CP05, CP14–CP16) + Postman Login y Cartelera
/api/reservas, Stub o formulario de pago	Crítico	Suite de compra (CP09–CP13) + Postman Reserva + JMeter
Dependencias, Express, Helmet o release	Crítico	Regresión total: Newman + 16 casos Selenium + JMeter

Optimización de recursos en el pipeline: Newman se ejecuta en cada push porque tarda menos de un segundo y protege el núcleo del negocio. La suite completa de Selenium y JMeter se reservan para merges a main y releases.

7. Reto 3: Comparativa de herramientas

Criterio	Selenium WebDriver	Cypress
Escenario CineFlex recomendado	**Negocio:** certificar la compra de boletos en Chrome, Firefox, Edge y Safari antes del	**Técnico:** pruebas rápidas del frontend durante el desarrollo diario (mapa de

Criterio	Selenium WebDriver	Cypress
	estreno, porque los clientes llegan desde cualquier navegador.	asientos, validaciones del formulario de pago).
Navegadores	Todos, incluido Safari, y ejecución en paralelo con Selenium Grid.	Chromium, Firefox y WebKit (experimental). Sin Safari real.
Lenguajes	JavaScript, Java, Python, C#, Ruby.	Solo JavaScript y TypeScript.
Arquitectura	Controla el navegador desde fuera con el protocolo W3C WebDriver.	Se ejecuta dentro del navegador, junto a la aplicación.
Esperas	Explícitas (driver.wait / until).	Automáticas en cada comando.
Pestañas y dominios	Soporta varias pestañas y dominios: útil si la pasarela real redirige al banco (3-D Secure).	Limitado: no maneja varias pestañas y el cambio de dominio requiere cy.origin.
Depuración	Logs y capturas configuradas a mano.	Time-travel debugging, capturas y video automáticos.
Decisión en CineFlex	Suite oficial de regresión (16 casos en ui_tests/).	Complemento opcional para el equipo de frontend.

Conclusión: para la certificación previa al estreno se usa Selenium WebDriver, porque el riesgo de negocio está en que la compra falle en algún navegador real y porque la futura pasarela bancaria puede redirigir a otro dominio. Cypress se recomienda para el ciclo de desarrollo diario del frontend, donde la velocidad de retroalimentación vale más que la cobertura de navegadores.

8. Plan de ejecución de pruebas

Nivel / Tipo	Herramienta	Cuándo se ejecuta	Artefacto
Integración (Stub)	Postman + Selenium	En cada push y en cada cambio de pagos	api_tests/, ui_tests/
API funcional y negativa	Postman / Newman	Automático en cada push (GitHub Actions)	CineFlex_API_Collection.json
Sistema / E2E de UI	Selenium + POM	Antes de cada merge a main y release	test_suite.js
Carga y estrés	Apache JMeter	Antes del release del estreno	prueba_carga.jmx
Regresión	Según la matriz de la sección 6.2	Según el tipo de cambio	Toda la suite o una parte

8.1 Casos de prueba de API

Endpoint	Aserciones
1. Login (POST)	Status 200 · Content-Type JSON · JWT de 3 partes · Helmet activo · guarda el token en el environment
2. Cartelera (GET)	Status 200 · JSON válido · SLA < 300 ms · cada película tiene id, título y género
3. Reserva negativa (POST)	Status 400/402 · mensaje "Pago rechazado: tarjeta inválida" · sin ticket · Content-Type JSON

8.2 Casos de prueba de UI (Selenium + POM)

ID	Caso de prueba	Módulo
CP01	Muestra la vista de inicio de sesión	Login
CP02	Credenciales inválidas muestran error con contador	Login
CP03	Campos vacíos muestran mensaje de obligatorios	Login
CP04	Bloqueo preventivo tras 3 intentos fallidos	Login
CP05	Login exitoso renderiza el título "Cartelera"	Login
CP06	Lista al menos 3 películas	Cartelera
CP07	Cada película tiene título y botón Comprar	Cartelera
CP08	Botón "Cerrar sesión" visible tras el login	Cartelera
CP09	Seleccionar película abre el mapa de asientos correcto	Reserva
CP10	Los asientos ocupados están deshabilitados	Reserva
CP11	"Continuar al pago" solo se habilita tras elegir asiento	Reserva
CP12	Tarjeta falsa rechazada con mensaje de error	Pago
CP13	E2E: compra completa muestra el ticket virtual	Pago
CP14	"Cerrar sesión" regresa al login	Seguridad
CP15	Botón "Atrás" tras cerrar sesión no muestra la cartelera	Seguridad
CP16	Acceso directo a /#/cartelera sin sesión redirige al login	Seguridad

Reglas del patrón POM aplicadas: las clases LoginPage, CarteleraPage y ReservaPage solo contienen localizadores (IDs y selectores CSS, sin XPath) y métodos de acción. Todas las aserciones están en test_suite.js.

9. Resultados obtenidos

Suite	Resultado	Estado
Postman / Newman (API)	12 de 12 aserciones aprobadas, 0 fallidas	Aprobado
Selenium WebDriver (UI)	16 de 16 casos aprobados	Aprobado
JMeter: 500 VUs, ramp-up 15 s	Error %: _____ *Throughput:* _____ req/s Tiempo promedio: _____ ms	_____
GitHub Actions (Newman en CI)	Pipeline en verde con bloqueo de merge en main	Aprobado

10. Riesgos y mitigación

Riesgo	Impacto	Mitigación
La API real del banco responde distinto a lo que asume el Stub	Alto	Prueba de contrato cuando exista el sandbox; el Stub está aislado en una sola función.
Saturación del servidor en el estreno	Crítico	Prueba de carga con JMeter antes del release y monitoreo del porcentaje de error.
Pruebas de UI frágiles por cambios visuales	Medio	Localizadores por ID y CSS centralizados en el POM: un cambio se corrige en un solo lugar.
Integrar código que rompe la API	Alto	Newman en cada push y regla de protección de rama en main.
Datos de prueba agotados (asientos ocupados)	Bajo	Datos en memoria que se reinician al levantar el backend.

11. Criterios de entrada y salida

11.1 Criterios de entrada

- El backend levanta sin errores y responde en /api/health.
- El frontend carga en http://localhost:3000 con todos los IDs definidos.
- Los datos de prueba (usuario y tarjetas) están disponibles.

11.2 Criterios de salida (listo para producción)

- 100 % de las aserciones de Newman aprobadas en el pipeline.
- 16 de 16 casos Selenium aprobados, incluidos el E2E y la navegación segura.
- JMeter con 500 usuarios: error menor al 1 % y throughput documentado.

- Cabeceras de Helmet presentes y trazabilidad de Morgan activa.

12. Conclusiones

- El Stub de la pasarela permitió certificar el flujo de venta sin esperar al banco, aplicando integración Top-Down.
- La regresión selectiva basada en riesgo evita ejecutar suites costosas ante cambios cosméticos, sin descuidar los flujos críticos.
- Selenium con POM da una suite mantenible y multinavegador para el estreno; Cypress queda como apoyo para el desarrollo diario.
- La combinación de Newman en CI, Selenium y JMeter cubre la calidad funcional, de seguridad y de rendimiento de CineFlex.

13. Anexos

Recurso	Enlace
Repositorio en GitHub	_____
Video de sustentación	_____
Pipeline (GitHub Actions)	_____

Estructura del repositorio:

```
qa-cineflex-project/
├── README.md
├── estrategia_pruebas.pdf
├── .github/workflows/qa-pipeline.yml
├── backend/        server.js, package.json
├── frontend/       index.html, styles.css
├── api_tests/      CineFlex_API_Collection.json, CineFlex_Environment.json
├── ui_tests/       pages/ (LoginPage, CarteleraPage, ReservaPage), test_suite.js
└── performance/    prueba_carga.jmx, summary_report.png
```